TECHNICAL AND PLAIN-LANGUAGE COMPANION

How Weeks 4-6 Work

The durable notification workflow, DLQ recovery code, Vault-backed runtime, and paired blue/green release system explained from first request to rollback.

Primary audience
Presenter, reviewer, and developer learning the system.

Source baseline
Current workspace on 24 August 2026.

Code convention
Numbers beside excerpts are current source line numbers.

Guarantee language
Durable admission + at-least-once transport + deduplicated committed effects.

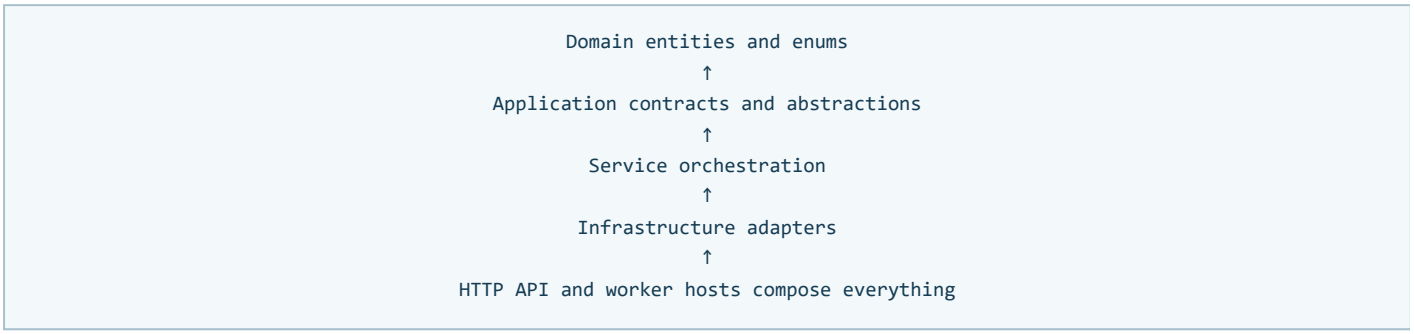
Shortest explanation: the API writes an instruction into the same SQL transaction as the job. A relay carries it to RabbitMQ. A worker records a receipt with its effects. Repeated deliveries are safe. Commands that repeatedly fail are quarantined, checked, delayed, and replayed only when the database says recovery is valid.

1. A layman’s mental model

System part	Everyday analogy	Actual responsibility
API	Receptionist	Checks the request and records an accepted job.
SQL transaction	Official ledger	Commits the job, items, idempotency receipt, and outgoing intent together.
Outbox	Sealed outgoing tray	Stores a message that still needs to reach RabbitMQ.
Outbox Relay	Courier	Leases due tray items, publishes with confirmation, and records progress.
RabbitMQ	Sorting office	Durably routes at-least-once commands to consumers.
Primary Worker	Fulfilment clerk	Creates notification effects and updates item/job state.
Inbox	Paid/processed stamp	Prevents a repeated message from committing the same effect twice.
DLQ	Quarantine shelf	Holds commands rejected after bounded processing attempts.
Recovery Worker	Safety inspector	Checks origin, format, replay budget, and database state.
Recovery queue	Cooling-off room	Waits five seconds, then routes an eligible command back.
Parking queue	Evidence locker	Retains messages unsafe for automatic replay with a reason.
Vault	Locked key cabinet	Gives each role access to only its secret drawer.
Blue/green + Nginx	Two storefronts, one sign	Checks an idle API/Next pair before moving stable traffic.

> **The recovery safety rule:** the inspector never throws away the quarantined parcel until RabbitMQ confirms that either a safe replay copy or a parked copy has been accepted.

Clean Architecture placement



The hosts know how to start processes. Domain/application code does not know about controllers, Docker, RabbitMQ clients, or Nginx.

2. End-to-end state and guarantee

```
POST + idempotency key → one SQL commit → 202 Accepted
                                  ↓
Outbox lease → confirmed broker publish → manual consumer delivery
                                  ↓
                effects + Inbox in one transaction → ACK
```

Normal path

1. The controller creates a version/operation/actor idempotency scope.
2. The service normalizes the body and hashes its semantic meaning.
3. A serializable transaction locks that scoped key and checks for replay or conflict.
4. The transaction inserts job, items, Outbox command, and original response receipt.
5. The API returns `202` after SQL commits; it never publishes to RabbitMQ directly.
6. The relay leases the Outbox row and publishes with a stable message ID.
7. The worker commits effects, item/job state, and Inbox receipt before ACK.

Failure/recovery path

```
processing error → retry publish #1 → retry publish #2 → persist DeadLettered → reject
                                  ↓
DLQ → eligibility checks → persist RecoveryPending → confirmed recovery publish → ACK
                                  ↓
            TTL expires → main queue → process pending items → Completed
```

What is and is not guaranteed

True	Not claimed
A returned 202 has a durable SQL job and outgoing intent.	RabbitMQ delivery is exactly once.
An identical scoped key/body replays the original response.	SQL commit and broker publish are one atomic transaction.
Inbox/item state suppress repeated committed database effects.	An external provider automatically deduplicates unless its contract proves it.
Unsettled consumer failures return to RabbitMQ.	Every parked message is automatically reconciled.
Rollback restores the prior API/Next pair.	Rollback reverses database migrations or shared state.

> **Why “at least once” matters:** a broker confirmation can succeed just before the relay crashes, or an ACK can be lost after SQL commits. The same stable command may appear again. Correctness comes from durable identities and deduplication, not from pretending duplication cannot happen.

3. Week 4 admission: controller code

Source: `notification-api/Presentation/Controllers/Notification/NotificationsController.cs:108-141`

```
108 [HttpPost("bulk")]
109 [ProducesResponseType(typeof(BulkNotificationJobDto), StatusCodes.Status202Accepted)]
111 [ProducesResponseType(typeof(ProblemDetails), StatusCodes.Status409Conflict)]
113 public async Task<ActionResult<BulkNotificationJobDto>> CreateBulkNotifications(
114     CreateBulkNotificationsRequest request,
115     [FromServices] IBulkNotificationJobService bulkJobs,
116     [FromHeader(Name = IdempotencyContext.HeaderName)] string? idempotencyKey,
117     CancellationToken cancellationToken)
118 {
119     var correlationId = HttpContext.TraceIdentifier;
120     var requestedVersion = HttpContext.GetRequestedApiVersion();
121     var idempotency = IdempotencyContext.Create(
122         $"notifications.bulk.create.v{requestedVersion?.MajorVersion ?? 2}",
123         "global",
124         idempotencyKey);
125     var result = await bulkJobs.QueueAsync(
126         request, correlationId, idempotency, cancellationToken);
130     var job = result.Value;
131     if (result.Replayed)
132     {
133         Response.Headers[IdempotencyContext.ReplayHeaderName] = "true";
134     }
136     Response.Headers["X-Correlation-ID"] = job.CorrelationId;
137     var statusLocation = requestedVersion is null
138         ? $"/api/notifications/bulk/{job.JobId}"
139         : $"/api/v{requestedVersion.MajorVersion}/notifications/bulk/{job.JobId}";
141     return Accepted(statusLocation, job);
}
```

Line(s)	Technical explanation	Plain language
108-112	Declares the bulk route and documented response contracts.	The receptionist displays the possible outcomes before taking a form.
113-117	Accepts the validated body, service abstraction, optional idempotency header, and cancellation.	Receive the form, repeat-ticket number, and stop signal.
119-120	Captures trace and requested API version.	Stamp the request with tracking and rulebook version.
121-124	Scopes the key by operation/version and actor boundary.	The same ticket is unique only at the correct service desk.
125-129	Delegates all durable admission to the application service.	The receptionist asks the ledger clerk to record it safely.
131-134	Marks a replay without changing the stored response.	Tell the caller this is the same receipt, not a new order.
136-141	Returns correlation, status URL, job body, and HTTP 202.	Give the caller a tracking link after the ledger is committed.

4. Week 4 admission: one SQL transaction

Source: `BulkNotificationJobService.cs:47-73, 83-140`

```
47  var normalizedItems = request.Notifications.Select(...).ToArray();
53  var requestHash = IdempotencyRequestHasher.Hash(...);
62  await using var transaction =
        await idempotencyStore.BeginSerializableTransactionAsync(cancellationToken);
63  await idempotencyStore.PurgeExpiredAsync(idempotency, cancellationToken);
64  await idempotencyStore.AcquireLockAsync(idempotency, cancellationToken);
65  var replay = await idempotencyStore.TryReplayAsync<BulkNotificationJobDto>(
        idempotency, requestHash, cancellationToken);
69  if (replay is not null)
70  {
71      await CommitAsync(transaction, cancellationToken);
72      return replay;
73  }

83  var now = timeProvider.GetUtcNow();
84  var jobId = Guid.NewGuid();
85  var messageId = Guid.NewGuid();
89  var job = new BulkNotificationJob
90  {
91      Id = jobId,
92      Status = BulkNotificationJobStatuses.Queued,
97      Items = normalizedItems.Select((item, index) => new BulkNotificationJobItem
98      {
99          Sequence = index,
105         Status = BulkNotificationItemStatuses.Pending
106     }).ToList()
107 };
109 jobs.Add(job);
110 var command = new BulkNotificationRequestedV1(... messageId, job.Id, ...);
116 dbContext.OutboxMessages.Add(new OutboxMessage
117 {
118     Id = messageId,
120     AggregateId = job.Id.ToString("D"),
123     Payload = JsonSerializer.Serialize(command, SerializerOptions),
124     Exchange = RabbitMqConfiguration.GetRequiredString(configuration, "CommandExchange"),
125     RoutingKey = RabbitMqConfiguration.GetRequiredString(configuration, "CommandRoutingKey")
129 });
131 var response = ToDto(job);
132 idempotencyStore.Add(idempotency, requestHash, StatusCodes.Status202Accepted,
        response, nameof(BulkNotificationJob), job.Id.ToString("D"));
139 await jobs.SaveChangesAsync(cancellationToken);
140 await CommitAsync(transaction, cancellationToken);
```

Line(s)	What the block guarantees
47-60	Equivalent input has a stable semantic hash despite harmless whitespace/case normalization choices.
62-68	A serializable transaction and transaction-owned key lock make concurrent same-key requests line up.
69-73	An exact replay returns the stored response; a different hash is rejected by the idempotency store.
83-107	Creates the SQL job and ordered pending items with stable identities.
110-129	Creates a small command and a durable Outbox row. Recipient/body details remain in SQL.
131-140	Stores the original 202 response and commits everything once. No RabbitMQ call is in this transaction.

> **Plain language:** the ledger records the work, outgoing slip, and customer receipt under one pen stroke. Either all of them exist or none do.

5. Outbox Relay and Inbox

Sources: `notification.outbox.worker/OutboxRelayService.cs` and `BulkNotificationProcessor.cs`

```
// OutboxRelayService.cs:41-51, 60-68
41 var now = timeProvider.GetUtcNow();
42 var candidateIds = await dbContext.OutboxMessages
44     .Where(message => message.PublishedAtUtc == null
45         && message.NextAttemptAtUtc <= now
46         && (message.LockedUntilUtc == null || message.LockedUntilUtc <= now))
47     .OrderBy(message => message.NextAttemptAtUtc)
50     .Take(options.BatchSize)
51     .ToListAsync(cancellationToken);
60 affected = await dbContext.OutboxMessages
61     .Where(message => message.Id == candidateId && ...)
65     .ExecuteUpdateAsync(setters => setters
66         .SetProperty(message => message.LockedBy, workerId)
67         .SetProperty(message => message.LockedUntilUtc, lockedUntil), ...);

// OutboxRelayService.cs:110-138
110 try
111 {
112     await publisher.PublishAsync(message, cancellationToken);
113     message.PublishedAtUtc = timeProvider.GetUtcNow();
114     message.LockedBy = null;
117     await dbContext.SaveChangesAsync(cancellationToken);
118     return true;
124 catch (Exception exception)
125 {
126     message.AttemptCount++;
127     message.NextAttemptAtUtc = timeProvider.GetUtcNow().Add(CalculateBackoff(message));
128     message.LockedBy = null;
130     message.LastError = $"{exception.GetType().Name}: broker publication failed.";
131     await dbContext.SaveChangesAsync(CancellationToken.None);
138     return false;
}
```

Line(s)	Explanation
41-51	Select due, unpublished, unlocked/expired rows in a stable order and bounded batch.
60-68	Conditionally claims each row. Competing relays cannot both own the same valid lease.
112-118	Publish with confirmation first; then record publication and release the lease.
124-138	On failure, preserve the row, increment attempts, schedule bounded backoff/jitter, and try later.

```
// BulkNotificationProcessor.cs:60-80, 181-196
60 if (messageId != Guid.Empty)
62     var receipt = await dbContext.InboxMessages.SingleOrDefaultAsync(...);
66     if (receipt is not null)
79         return BulkNotificationProcessingResult.Completed;
181 if (messageId != Guid.Empty)
183     dbContext.InboxMessages.Add(new InboxMessage {
185         MessageId = messageId,
186         Handler = BulkNotificationRequestedV1.MessageType,
187         AggregateId = jobId
189     });
192 await jobs.SaveChangesAsync(cancellationToken);
195 await transaction.CommitAsync(cancellationToken);
```

> **Duplicate boundary:** RabbitMQ can confirm at line 112 and the relay can crash before line 117. A repeated stable message is expected; the Inbox detects it before repeating effects.

6. Primary worker: ACK ordering

Source: `notification.worker/Handlers/BulkNotificationCommandHandler.cs:22-125`

```
22 public async Task HandleAsync(IRabbitMqDelivery delivery, CancellationToken ct)
23 {
24     var body = delivery.Body.ToArray();
25     if (!BulkNotificationCommandReader.TryRead(body, delivery.Properties,
26             out var command, out var invalidReason))
27     {
31         await delivery.RejectAsync(requeue: false, ct);
32         return;
33     }
35     var retryCount = RabbitMqMessageHeaders.GetInt32(
           delivery.Properties.Headers, RetryHeader);
37     try
38     {
39         await using var scope = scopeFactory.CreateAsyncScope();
40         var processor = scope.ServiceProvider
               .GetRequiredService<BulkNotificationProcessor>();
41         processingResult = await processor.ProcessAsync(
42             command.MessageId, command.JobId, ct);
45     }
50     catch (Exception exception)
51     {
52         await HandleProcessingFailureAsync(
               delivery, body, command, retryCount, exception, ct);
53         return;
54     }
56     if (processingResult == BulkNotificationProcessingResult.AlreadyDeadLettered)
58         await delivery.RejectAsync(requeue: false, ct);
66     await delivery.AcknowledgeAsync(ct);
}
```

```
// HandleProcessingFailureAsync:82-125
82 var maxAttempts = RabbitMqConfiguration.GetInt32(... "MaxDeliveryAttempts", 1, 20);
83 if (retryCount >= maxAttempts - 1)
84 {
85     await UpdateJobForDeadLetterAsync(command.JobId, maxAttempts, ct);
93     await delivery.RejectAsync(requeue: false, ct);
94     return;
95 }
97 var completedAttempts = retryCount + 1;
98 await UpdateJobForRetryAsync(command.JobId, completedAttempts, maxAttempts, ct);
99 var retryProperties = CreateRetryProperties(delivery.Properties, completedAttempts);
102 await delivery.PublishAsync(commandExchange, commandRoutingKey,
       retryProperties, body, ct);
118 await delivery.AcknowledgeAsync(ct);
```

Line(s)	Why the order matters
25-32	Malformed/unsupported commands cannot become valid by retry; reject to DLQ for classification.
39-44	The processor commits effects and Inbox before returning success.
50-53	Expected processing failures enter the explicit bounded-retry path.
83-94	On the final attempt, persist `DeadLettered` before rejecting to the DLQ.
97-118	For a retry, persist retry state, confirm-publish replacement, then ACK the original.

> **Plain language:** never throw away the old work ticket until the replacement ticket is safely accepted. On the final failure, update the official ledger before putting the ticket on the quarantine shelf.

7. DLQ worker bootstrap and consumer

Sources: `notification.dlq.worker/Program.cs` and `Consumers/RabbitMqDeadLetterRecoveryWorker.cs`

```
// Program.cs:9-18
 9 var builder = Host.CreateApplicationBuilder(args);
10 builder.Configuration.AddKeyPerFile("/run/secrets", optional: true);
12 builder.Services.AddNotificationWorkerInfrastructure(builder.Configuration);
13 builder.Services.AddScoped<IBulkNotificationJobRepository,
       BulkNotificationJobRepository>();
14 builder.Services.AddScoped<BulkNotificationDeadLetterRecoveryService>();
15 builder.Services.AddSingleton<DeadLetterRecoveryDeliveryHandler>();
16 builder.Services.AddHostedService<RabbitMqDeadLetterRecoveryWorker>();
18 await builder.Build().RunAsync();

// RabbitMqDeadLetterRecoveryWorker.cs:11-34
11 public sealed class RabbitMqDeadLetterRecoveryWorker(
12     DeadLetterRecoveryDeliveryHandler handler,
13     IServiceScopeFactory scopeFactory,
14     IConfiguration configuration,
15     ILogger<RabbitMqDeadLetterRecoveryWorker> logger)
16     : RabbitMqConsumerWorkerBase(
17         scopeFactory, configuration, logger,
20         readinessFileName: "nsa-dlq-recovery-worker-ready")
21 {
22     protected override string ConsumerName => "nsa-dlq-recovery-worker";
24     protected override string QueueName => RabbitMqValue("DeadLetterQueue");
28     protected override ushort PrefetchCount => 1;
30     protected override Task HandleDeliveryAsync(
31         IChannel channel, BasicDeliverEventArgs eventArgs, CancellationToken ct) =>
34         handler.HandleAsync(new RabbitMqDelivery(channel, eventArgs), ct);
}
```

Line(s)	Technical	Plain language
P9-10	Creates the host and maps secret filenames to configuration keys.	Start the inspector and read only its sealed envelopes.
P12	Registers shared DB, RabbitMQ, topology, and processing infrastructure.	Give the inspector its desk, ledger access, and post-office connection.
P13-16	Registers repository, recovery state service, delivery handler, and hosted consumer.	Assign specialists and start the long-running shift.
W16-20	Reuses common connection/requeue/readiness behavior.	Use the same safe loading dock as the normal worker.
W24	Consumes only the configured DLQ.	Inspect only the quarantine shelf.
W28	Prefetch one keeps recovery conservative and ordered.	Inspect one parcel at a time.
W30-34	Wraps the RabbitMQ delivery behind an abstraction and delegates decisions.	Pass the parcel to the rule-checker.

8. Recovery handler: classify the delivery

Source: `DeadLetterRecoveryDeliveryHandler.cs:19-94`

```
19 public async Task HandleAsync(IRabbitMqDelivery delivery, CancellationToken ct)
20 {
21     var body = delivery.Body;
22     if (!DeadLetterRecoveryMessageInspector.WasRejectedFromCommandQueue(
23             delivery.Properties,
24             RabbitMqConfiguration.GetRequiredString(configuration, "CommandQueue")))
25     {
26         await MoveToParkingLotAsync(delivery, body,
29             "not-a-rejected-command-delivery", ct);
31         return;
32     }
34     if (!DeadLetterRecoveryMessageInspector.TryReadCommand(
35             body, delivery.Properties, out var command, out var invalidReason))
39     {
44         await MoveToParkingLotAsync(delivery, body, invalidReason, ct);
45         return;
46     }
48     var replayCount = DeadLetterRecoveryMessageInspector.ReadHeaderCount(
49         delivery.Properties.Headers,
50         DeadLetterRecoveryMessageInspector.ReplayCountHeader);
51     var maxReplayAttempts = RabbitMqConfiguration.GetInt32(
52         configuration, "MaxDeadLetterReplayAttempts", 1, 20);
56     if (replayCount >= maxReplayAttempts)
57     {
62         await MoveToParkingLotAsync(
             delivery, body, "replay-limit-exhausted", ct);
63         return;
64     }
66     await using var scope = scopeFactory.CreateAsyncScope();
67     var recovery = scope.ServiceProvider
         .GetRequiredService<BulkNotificationDeadLetterRecoveryService>();
68     var recoveryResult =
         await recovery.PrepareForReplayAsync(command.JobId, ct);
70     switch (recoveryResult)
71     {
72         case BulkNotificationDeadLetterRecoveryResult.NoRecoveryRequired:
73             await delivery.AcknowledgeAsync(ct);
74             return;
76         case BulkNotificationDeadLetterRecoveryResult.JobNotFound:
77             await MoveToParkingLotAsync(delivery, body, "unknown-job", ct);
78             return;
80         case BulkNotificationDeadLetterRecoveryResult.RecoveryPrepared:
81             await ReplayAsync(delivery, body, command,
85                 replayCount, maxReplayAttempts, ct);
88             return;
90         default:
91             throw new InvalidOperationException(...);
93     }
94 }
```

9. Recovery handler: line-by-line meaning

Line(s)	Technical explanation	Layman's explanation
19-21	Accept one unsettled delivery and retain its exact body.	Pick up one quarantined parcel without discarding it.
22-24	Require broker death metadata proving rejection from the configured main queue.	Verify it truly came from the approved sorting line.
25-31	Wrong-origin messages go to parking with a machine-readable reason.	Unknown parcels go to the evidence shelf, not back into circulation.
34-38	Deserialize and validate schema/type/body into the versioned command.	Open the label and check that it is readable and supported.
39-45	Malformed/unsupported commands are parked.	An unreadable address needs a human, not another delivery attempt.
48-55	Read independent DLQ replay count and configured ceiling.	Count how many times the inspector already retried this parcel.
56-63	Park commands that exhausted automatic recovery.	Stop an endless loop and preserve the evidence.
66-68	Create a scoped SQL service and ask the database whether replay is valid.	Consult the official ledger before moving anything.
70-74	Completed/irrelevant state needs no replay; ACK removes the stale DLQ copy.	The order is already done, so file the duplicate ticket.
76-78	An unknown job is parked, retaining its body and identity.	No ledger entry means the parcel needs investigation.
80-88	Only `RecoveryPrepared` reaches confirmed recovery publication.	Only an approved parcel returns to delivery.
90-92	Unexpected enum values fail loudly; the shared host requeues the unsettled delivery.	An unknown rule never silently throws the parcel away.

> **Central design:** validation is layered. Broker origin, message format, replay budget, and persisted business state must all agree before automatic replay.

Why a parking queue is different from a DLQ

The DLQ contains commands that followed the known processing path and exhausted normal attempts. Parking contains commands the recovery system cannot safely reason about: wrong origin, malformed/unsupported body, unknown job, or exhausted recovery attempts.

10. Replay and parking: publish before ACK

Source: `DeadLetterRecoveryDeliveryHandler.cs:96-144`

```
96  private async Task ReplayAsync(
97      IRabbitMqDelivery delivery,
98      ReadOnlyMemory<byte> body,
99      BulkNotificationRequestedV1 command,
100     int replayCount,
101     int maxReplayAttempts,
102     CancellationToken ct)
103 {
104     var replayProperties =
105         DeadLetterRecoveryMessageInspector.CreateReplayProperties(
106             delivery.Properties, replayCount + 1);
107     await delivery.PublishAsync(
108         RabbitMqConfiguration.GetRequiredString(configuration, "RecoveryExchange"),
109         RabbitMqConfiguration.GetRequiredString(configuration, "RecoveryRoutingKey"),
110         replayProperties, body, ct);
113     await delivery.AcknowledgeAsync(ct);
121 }

123 private async Task MoveToParkingLotAsync(
124     IRabbitMqDelivery delivery,
125     ReadOnlyMemory<byte> body,
126     string reason,
127     CancellationToken ct)
128 {
129     var parkingProperties =
130         DeadLetterRecoveryMessageInspector.CreateParkingLotProperties(
131             delivery.Properties, reason);
132     await delivery.PublishAsync(
133         RabbitMqConfiguration.GetRequiredString(configuration, "ParkingLotExchange"),
134         RabbitMqConfiguration.GetRequiredString(configuration, "ParkingLotRoutingKey"),
135         parkingProperties, body, ct);
138     await delivery.AcknowledgeAsync(ct);
144 }
```

Line(s)	Explanation
96-102	Replay needs the exact original delivery/body, parsed identity, current replay budget, and cancellation.
104-106	Clone persistent identity/tracing metadata, clear exhausted normal counters, increment DLQ replay count.
107-112	Mandatory confirmed publish to the durable recovery exchange/queue. Failure throws.
113	ACK the DLQ copy only after the replacement publication succeeds.
123-131	Parking copies metadata and adds `x-dlq-recovery-reason`.
132-137	Confirm-publish to the durable parking route.
138	ACK only after the investigable copy is safe.

> **If publish fails:** neither path reaches ACK. The exception returns to the shared consumer host, which NACKs/requeues or closes the channel so RabbitMQ redelivers.

11. Message inspector and header reset

Source: `DeadLetterRecoveryMessageInspector.cs:20-87`

```
20 internal static bool TryReadCommand(
21     ReadOnlyMemory<byte> body,
22     IReadOnlyBasicProperties properties,
23     out BulkNotificationRequestedV1 command,
24     out string invalidReason)
25 {
26     if (!BulkNotificationCommandReader.TryRead(
           body, properties, out command, out var reason))
27     {
29         invalidReason =
           reason == BulkNotificationCommandInvalidReason.Malformed
30             ? "malformed-command"
31             : "unsupported-command";
32         return false;
33     }
35     invalidReason = string.Empty;
36     return true;
37 }

39 internal static bool WasRejectedFromCommandQueue(
       IReadOnlyBasicProperties properties, string commandQueue)
41 {
42     if (properties.Headers is null) return false;
46     if (TryGetString(headers, "x-last-death-reason", out var lastReason)
47         && TryGetString(headers, "x-last-death-queue", out var lastQueue))
49         return lastReason == "rejected" && lastQueue == commandQueue;
53     if (!headers.TryGetValue("x-death", out var deaths) || ...)
55         return false;
58     foreach (var death in entries)
60         if (death.reason == "rejected" && death.queue == commandQueue)
65             return true;
69     return false;
70 }

72 internal static BasicProperties CreateReplayProperties(
       IReadOnlyBasicProperties source, int replayCount)
74 {
75     var headers = RabbitMqMessageHeaders.Clone(source.Headers);
76     headers.Remove("x-retry-count");
77     headers.Remove("x-delivery-count");
78     headers.Remove("x-acquired-count");
79     headers["x-dlq-replay-count"] = replayCount;
80     return RabbitMqMessageProperties.CreatePersistentCopy(source, headers);
81 }
82 internal static BasicProperties CreateParkingLotProperties(..., string reason)
84 {
85     headers["x-dlq-recovery-reason"] = reason;
86     return RabbitMqMessageProperties.CreatePersistentCopy(source, headers);
87 }
```

Lines	Why
20-37	Distinguishes malformed JSON from an unsupported contract; operators get a useful reason.
39-70	Supports RabbitMQ's short last-death headers and full `x-death` history, but requires reason + exact queue.
75-79	Normal attempt and broker counters are already exhausted. Remove them so recovery gets a fresh bounded normal path; keep a separate recovery counter.
80,86	Persistent copy retains MessageId, type, correlation, and trace metadata.
82-86	Parked messages carry a machine-readable investigation reason.

12. SQL recovery state transition

Source: `BulkNotificationDeadLetterRecoveryService.cs:15-55`

```
15 public async Task<BulkNotificationDeadLetterRecoveryResult>
       PrepareForReplayAsync(Guid jobId, CancellationToken ct)
18 {
19     var job = await jobs.GetByIdAsync(jobId, ct);
20     if (job is null)
21     {
25         return BulkNotificationDeadLetterRecoveryResult.JobNotFound;
26     }
28     if (job.Status is BulkNotificationJobStatuses.Completed
29         or BulkNotificationJobStatuses.CompletedWithErrors)
30     {
34         return BulkNotificationDeadLetterRecoveryResult.NoRecoveryRequired;
35     }
37     if (job.Status == BulkNotificationJobStatuses.DeadLettered)
38     {
39         var now = timeProvider.GetUtcNow();
40         job.Status = BulkNotificationJobStatuses.RecoveryPending;
41         job.QueuedAtUtc = now;
42         job.StartedAtUtc = null;
43         job.CompletedAtUtc = null;
44         job.Error = "Requeued by the dead-letter recovery worker.";
45         await jobs.SaveChangesAsync(ct);
50     }
52     return job.Status == BulkNotificationJobStatuses.RecoveryPending
53         ? BulkNotificationDeadLetterRecoveryResult.RecoveryPrepared
54         : BulkNotificationDeadLetterRecoveryResult.NoRecoveryRequired;
55 }
```

Line(s)	Technical	Plain language
19-26	No job means there is no trustworthy aggregate to recover.	A parcel without a ledger entry goes to investigation.
28-35	Terminal jobs must not repeat effects; the stale DLQ delivery can be ACKed.	If the order is already done, discard only the duplicate ticket.
37-45	Only `DeadLettered` transitions to `RecoveryPending`; timestamps and operator-readable error are reset/saved.	The ledger explicitly reopens the job before another attempt.
52-54	Replay is allowed only when persisted state is now `RecoveryPending`.	The inspector needs written authorization, not a guess.

Why save before publish?

The normal worker can receive the replay very quickly. It must see a processable state. If SQL preparation fails, no replacement is published and the original DLQ delivery remains unsettled/recoverable.

> A publish may succeed and its subsequent ACK may be lost. A duplicate replay can occur. Terminal state, pending-item selection, and Inbox/message identity keep that duplicate from repeating completed work.

13. RabbitMQ topology: four queues, four jobs

Source: `RabbitMqTopology.cs:33-92`

```
33 await DeclareExchangeAsync(channel, commandExchange, ct);
34 await DeclareExchangeAsync(channel, deadLetterExchange, ct);
35 await DeclareExchangeAsync(channel, parkingLotExchange, ct);
36 await DeclareExchangeAsync(channel, recoveryExchange, ct);

38 await channel.QueueDeclareAsync(commandQueue,
40     durable: true, exclusive: false, autoDelete: false,
43     arguments: new Dictionary<string, object?> {
45         ["x-queue-type"] = "quorum",
46         ["x-delivery-limit"] = brokerDeliveryLimit,
47         ["x-dead-letter-exchange"] = deadLetterExchange,
48         ["x-dead-letter-routing-key"] = deadLetterRoutingKey
49     }, ...);
53 await BindAsync(channel, commandQueue, commandExchange, commandRoutingKey, ct);

55 await channel.QueueDeclareAsync(deadLetterQueue,
57     durable: true, ... arguments: QuorumQueueArguments(), ...);
64 await BindAsync(channel, deadLetterQueue,
       deadLetterExchange, deadLetterRoutingKey, ct);

66 await channel.QueueDeclareAsync(parkingLotQueue,
68     durable: true, ... arguments: QuorumQueueArguments(), ...);
75 await BindAsync(channel, parkingLotQueue,
       parkingLotExchange, parkingLotRoutingKey, ct);

77 await channel.QueueDeclareAsync(recoveryQueue,
79     durable: true, exclusive: false, autoDelete: false,
82     arguments: new Dictionary<string, object?> {
84         ["x-queue-type"] = "quorum",
85         ["x-message-ttl"] = replayDelayMilliseconds,
86         ["x-dead-letter-exchange"] = commandExchange,
87         ["x-dead-letter-routing-key"] = commandRoutingKey
88     }, ...);
92 await BindAsync(channel, recoveryQueue,
       recoveryExchange, recoveryRoutingKey, ct);
```

Queue	Purpose	Exit
Main command	Normal work; quorum + broker delivery safeguard.	ACK on success; explicit rejection routes to DLQ.
DLQ	Known rejected commands awaiting classification/recovery.	Confirmed replay or confirmed parking, then ACK.
Recovery	Durable cooling-off delay, default 5,000 ms.	TTL dead-letters back to main command route.
Parking	Unsafe automatic recovery; preserve body + reason.	Operator investigation/reconciliation only.

> **Why no sleep in the worker?** the broker owns the durable delay. A process restart does not forget a timer, and no consumer thread is held idle.

14. Week 5: Vault and full Compose runtime

KeyPerFile at every .NET host boundary

```
// API Program.cs:30-31 and each worker Program.cs
builder.Configuration.AddKeyPerFile("/run/secrets", optional: true);
```

A file named `RabbitMq__Password` maps to the nested configuration key `RabbitMq:Password`. The value never needs to appear in tracked JSON or a command-line argument.

Role/service	Secret boundary	Why
API AppRole	API DB connection + JWT signing key	Admits/queries data and issues/verifies access tokens; no broker credential.
Worker AppRole	Worker DB connection + RabbitMQ user/password	Relay and consumers need SQL and broker, but no JWT secret.
Migrator AppRole	Schema-owner DB connection	Only the one-shot migrator changes schema.

Provisioning sequence

1. Validate Docker, shell tools, Windows/NTFS ACL guard, runtime ownership, and persisted-state consistency.
2. Initialize/unseal persisted Vault or reuse matching protected recovery material.
3. Create policies/AppRoles and render files using short-lived role tokens, not the root token.
4. Assert cross-role reads fail.
5. Start SQL/RabbitMQ, reconcile ordinary app and schema-owner database logins, and run the one-shot migrator.
6. Start API, relay, primary worker, DLQ worker, Next, and optional Angular; wait for health.
7. Run end-to-end smoke. Only after success, establish/compare the Vault-held secret fingerprint and write non-secret evidence.

> **Layman's explanation:** Vault is a cabinet with three drawer badges. The provisioner uses each badge briefly to place sealed keys at the correct desk. A desk receives its key, not a master badge.

> On Windows/NTFS, Unix mode bits are not enough. The provisioner validates a Windows ACL restricted to the current user, SYSTEM, and Administrators. Never publish runtime secrets, AppRole material, Vault init/recovery material, or raw logs containing request bodies.

Why the second provision matters

It proves the process converges without resetting volumes or silently replacing credentials that persisted SQL/RabbitMQ state still expects. Evidence should show `vault_cross_role_reads=denied` and `secret_set_fingerprint_status=stable`.

15. Week 6: paired promotion code

Source: `infra/week6/deploy.sh:76-146` (condensed without changing order)

```
76  inspect and validate the release manifest
87  inspect current deployment state
91  choose the opposite idle color; first deployment chooses blue
97  write exact manifest images into release.env
98  validate the composed configuration
101 ensure exact API image exists
102 ensure exact Next image exists
103 ensure exact migrator image exists
105 run the compatible one-shot migrator
108 start idle API; wait healthy
110 start idle Next; wait healthy
113 create timestamped evidence path
115 run idle smoke; verify color and revision
117 preserve the active Nginx include as prior
122 write and validate candidate upstream include
127 start continuous former-to-target cutover probe
131 mark traffic switch in progress
132 atomically switch Nginx to candidate include
133 require the cutover probe to pass
141 run target-only observation probe
146 commit release state only after every gate passes
```

Stage	Why it exists
`Manifest`	API and Next are one immutable release unit; no target-host rebuild.
`Migrator`	Forward-compatible schema change runs once before new application traffic.
`Idle health`	A container is not promotable merely because it started; both processes must be healthy and identify correctly.
`Nginx validation`	Catch an invalid route before the active include changes.
`Continuous probe`	Prove requests survive the transition and observe only allowed release identities.
`Observation`	Prove stable traffic settles entirely on the target.
`State commit`	Only a verified cutover becomes the new rollback reference.

> **Plain language:** stock and inspect the closed storefront, test its doors, validate the sign mechanism, watch customers while flipping the sign, then write the official change in the release ledger.

Failure cleanup

If a post-switch gate fails before commit, the script restores the previous include and stops the uncommitted candidate. Evidence and candidate logs remain for diagnosis.

16. Week 6: rollback code and limits

Source: `infra/week6/rollback.sh:62-122` (condensed)

```
62  read current and previous release state
68  fail if no verified prior release exists
70  select the archived previous manifest (or exact supplied file)
76  fail if the manifest is unavailable
84  validate exact hash/revision and write rollback environment
89  ensure exact prior API image
90  ensure exact prior Next image
92  announce that no migration will run
93  start prior API; wait healthy
95  start prior Next; wait healthy
99  run idle smoke against the exact prior revision
102 preserve current active include
103 write and validate prior pair as candidate
105 run continuous rollback cutover probe
110 atomically switch Nginx
111 require transition probe success
114 run target-only observation
119 commit rollback state
122 report no migration or shared-state reversal
```

Why rollback does not reverse the database

Both colors share SQL. Automatically undoing schema while another process uses it is unsafe. Week 6 requires compatible forward migrations: the prior application must remain able to run during the rollback window.

Question	Answer
Does rollback rebuild?	No. It restores exact content-addressed images from the verified prior manifest.
Does it migrate?	No migration and no down migration.
Does it lose queued work?	No. SQL, Outbox, RabbitMQ, workers, Inbox, Vault, and volumes are shared and remain running.
Why keep the former color?	Warm fallback makes rollback fast and independently health-checkable.
When can it be retired?	Only after an explicit rollback-window decision; retirement is not part of the normal demo.

> **Storefront analogy:** rollback points the sign to the prior inspected storefront. It does not rewind the shared stockroom, so stockroom changes must remain compatible with both storefronts during the rollback window.

17. Failure matrix, tests, and presenter Q&A

Failure	System behavior
RabbitMQ down during HTTP request	SQL admission and 202 remain available; Outbox stays unpublished and retries later.
Relay crashes after broker confirm	Command can repeat; stable MessageId + Inbox suppress repeated effects.
Primary retry publication fails	Original remains unsettled; shared host NACKs/requeues.
Final SQL dead-letter update fails	No intentional reject; broker redelivery remains possible.
Recovery SQL preparation fails	No replay publish; original DLQ delivery requeues.
Replay/parking publish fails	No ACK; original DLQ message remains recoverable.
ACK lost after confirmed replay	Possible duplicate replay; state/item/Inbox checks make it safe.
Candidate release health fails	No cutover; idle candidate logs/evidence retained.
Post-switch gate fails before commit	Prior Nginx include restored automatically.

Executable evidence beside the design

- `Week4IdempotencyApiTests`: exact replay and changed-body conflict.
- `OutboxRelayServiceTests`: stable message identity, lease, retry/backoff.
- `BulkNotificationWorkerTests`: Inbox duplicate, poison retry, and dead-letter ordering.
- `BulkNotificationDeadLetterRecoveryTests`: eligible recovery, terminal ACK, unknown-job parking.
- `RabbitMqDeadLetterRecoveryWorkerTests`: death origin, header reset, replay/parking metadata.
- Live integration + Postman/Newman + Week 4 outage verifier: real SQL/RabbitMQ boundary.
- Week 5 smoke and Week 6 transition evidence: full platform and release control.

Short answers

Question	Answer
Why Outbox?	So accepted business state cannot exist without durable publication intent.
Why Inbox?	Because confirmed publish/mark and commit/ACK gaps can duplicate delivery.
Why ACK last?	ACK removes broker responsibility; the replacement or effect must be safe first.
Why reset retry headers?	Recovery is a separately bounded decision; exhausted normal counters would instantly dead-letter again.
DLQ versus parking?	DLQ is recoverable known failure; parking is unsafe/unknown automatic recovery.
Why no worker colors?	Week 6 releases the HTTP API/Next pair; shared workers continuously drain durable state.

> **Final takeaway:** reliability comes from explicit durable boundaries, stable identities, settlement order, bounded recovery, least-privilege runtime configuration, and fail-closed release gates.

Weeks 1-3 baseline: `docs/ADR-Week3.md`. Later decisions remain in the weekly records under `docs/adr`. Commands: `DLQ-Demonstration-Guide.pdf`.